

Lab 5: Build, Test, and Package Applications with IntelliJ IDEA and Docker



Name : Eman Khalid

Reg no : 23-BCS-026

Course Code: CS-649

Section: A

GitHub username: EmanKhalid27

Submitted To: Sir Waqas Saleem

Department: Computer Science

1. Introduction

This lab demonstrates how to use IntelliJ IDEA running inside a Docker container to build, test, and package applications. It covers cloning Maven, Gradle, Node.js, and React projects, installing build tools, resolving port conflicts, creating webpack bundles, and building Docker images.

2. Task Evidence

Task 1: Clone and Launch IntelliJ IDEA

```
emankhalid27@ubuntu:~/Intellij-Idea-docker$ git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git
Cloning into 'Intellij-Idea-docker'...
remote: Enumerating objects: 35, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 35 (delta 16), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (35/35), 15.24 KiB | 2.18 MiB/s, done.
Resolving deltas: 100% (16/16), done.
emankhalid27@ubuntu:~/Intellij-Idea-docker$ cd Intellij-Idea-docker
emankhalid27@ubuntu:~/Intellij-Idea-docker/Intellij-Idea-docker$ ls
Dockerfile  README.md  compose.yaml  init-terminal.sh  intellij-idea.desktop  remove.sh  resources.sh  setup.sh
emankhalid27@ubuntu:~/Intellij-Idea-docker/Intellij-Idea-docker$
```

Figure 1.1: Cloned the IntelliJ Docker repository. (intellij_repo_clone.png)

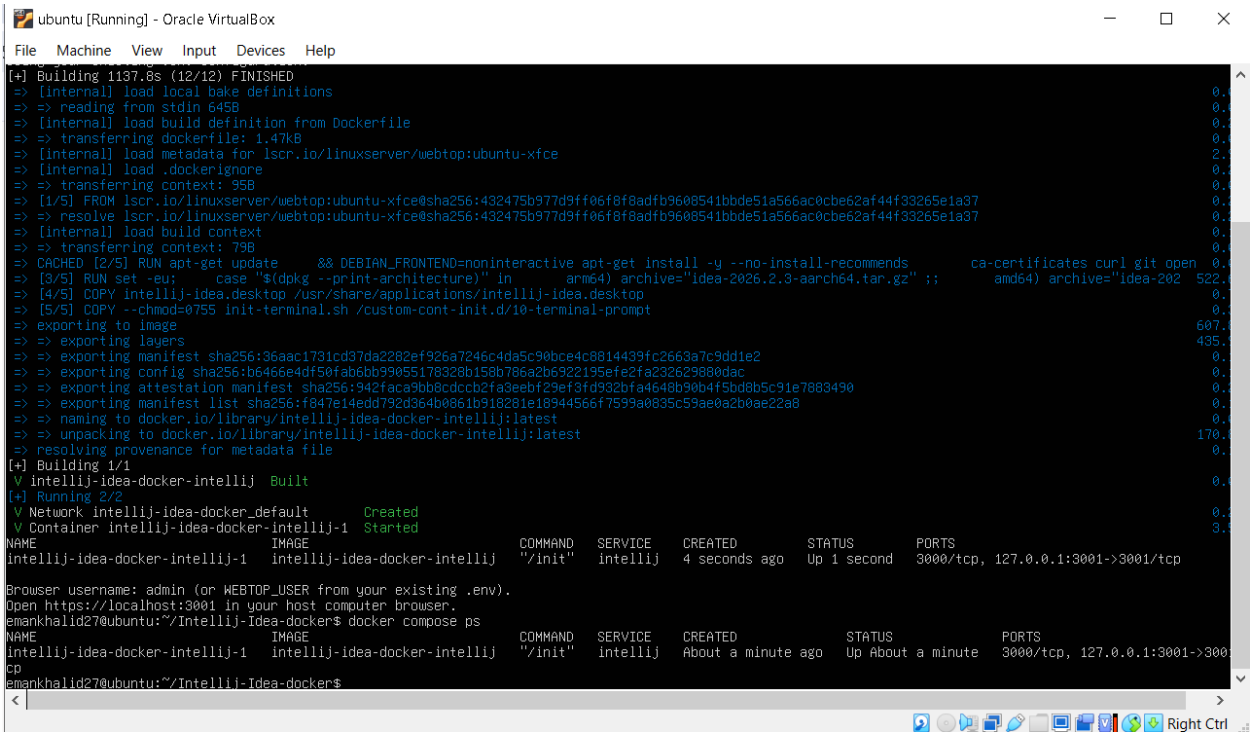


Figure 1.2: IntelliJ container running on port 3001. (intellij_running.png)

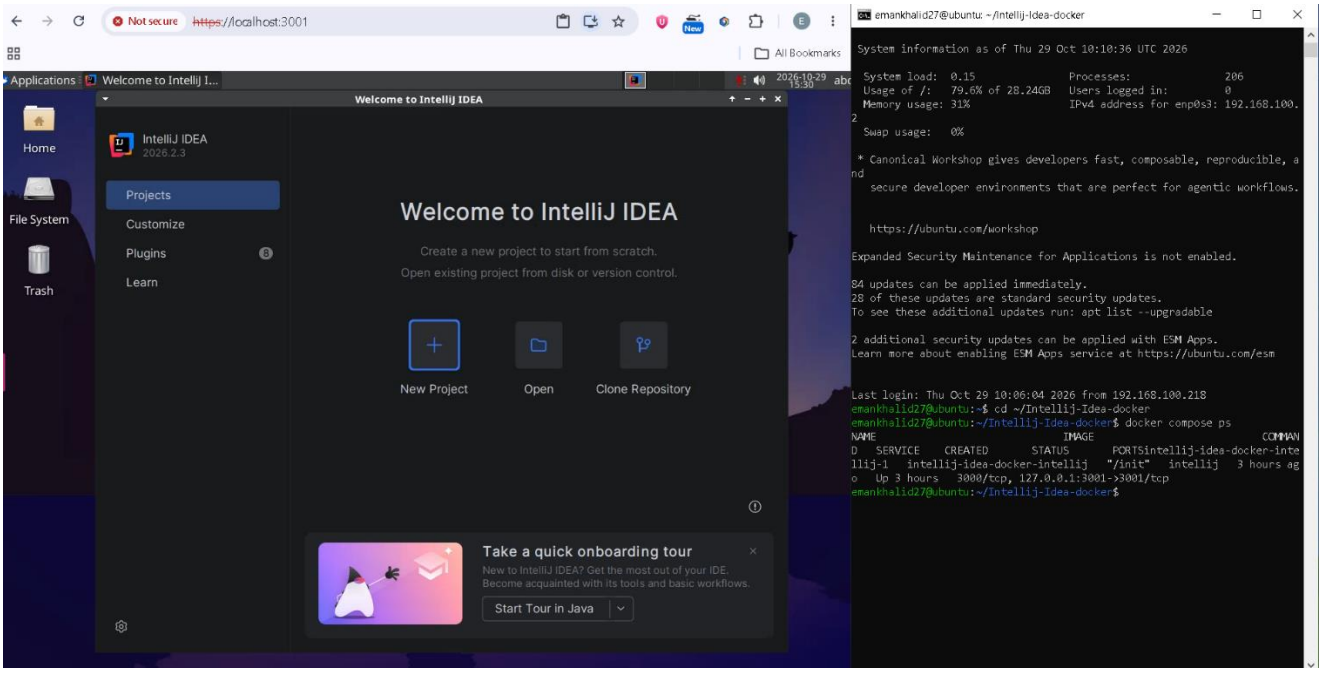


Figure 1.3: IntelliJ Welcome screen in the browser desktop. (intellij_desktop.png)

Task 2: Build and Run the Maven Project

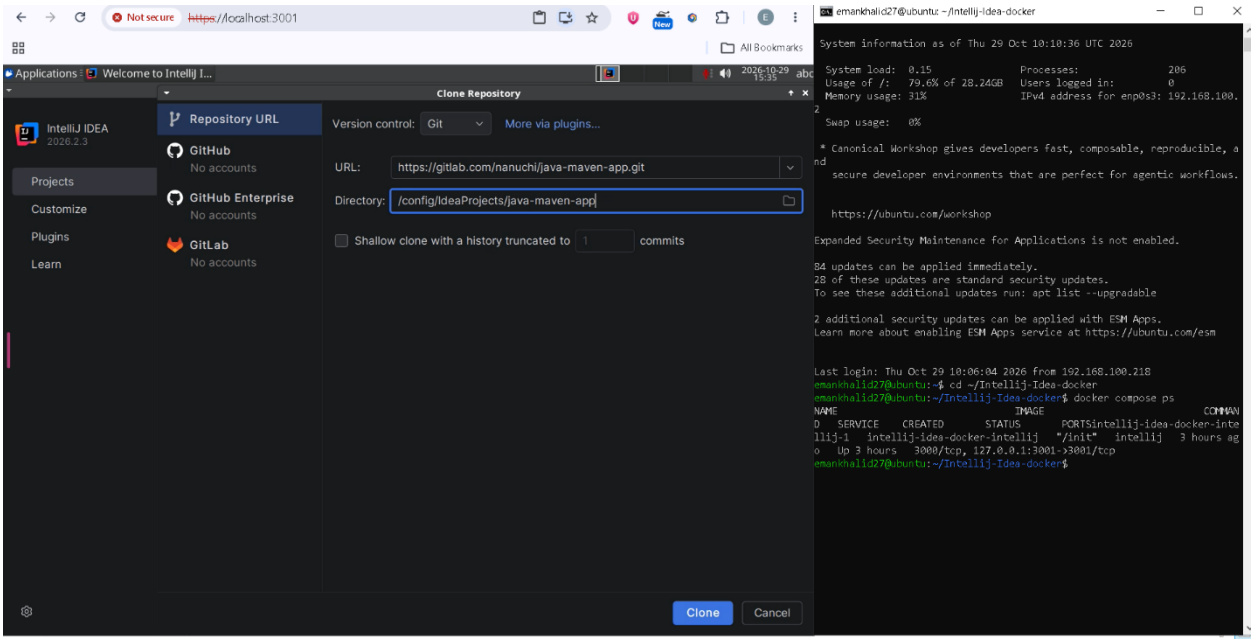


Figure 2.1: Cloning the Maven project. (maven_project_clone.png)

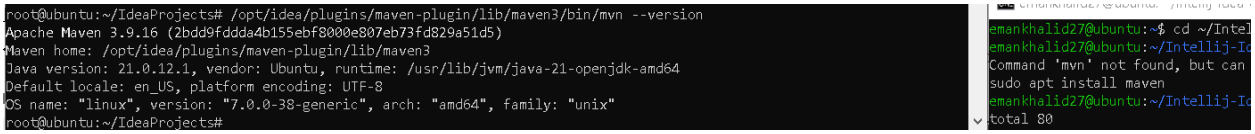


Figure 2.2: Maven version installed. (maven_installed.png)

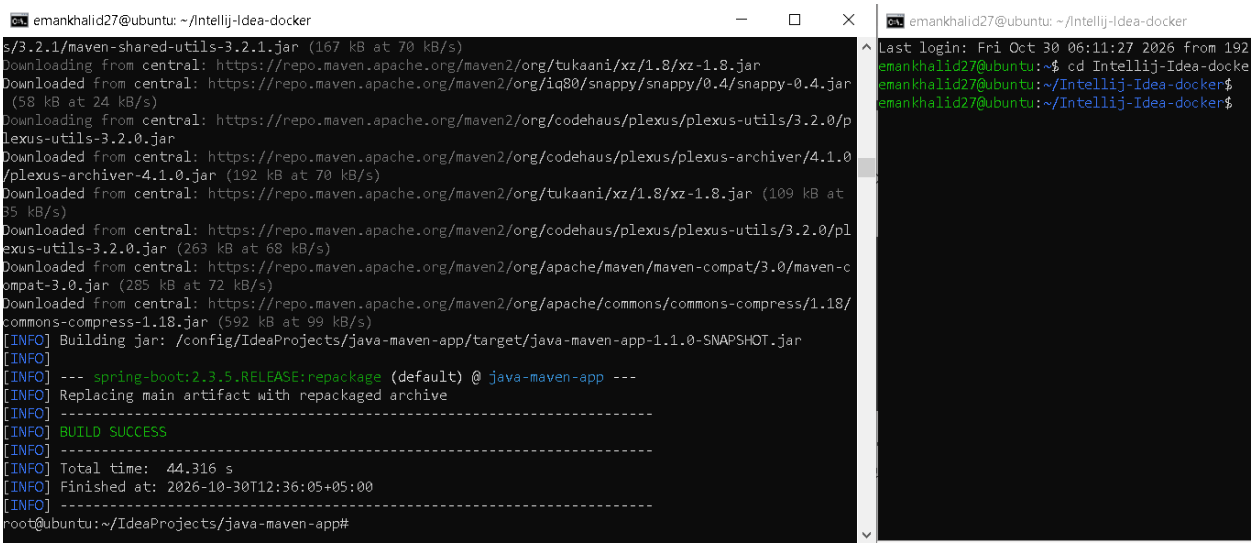


Figure 2.3: Successful Maven package build. (maven_package.png)

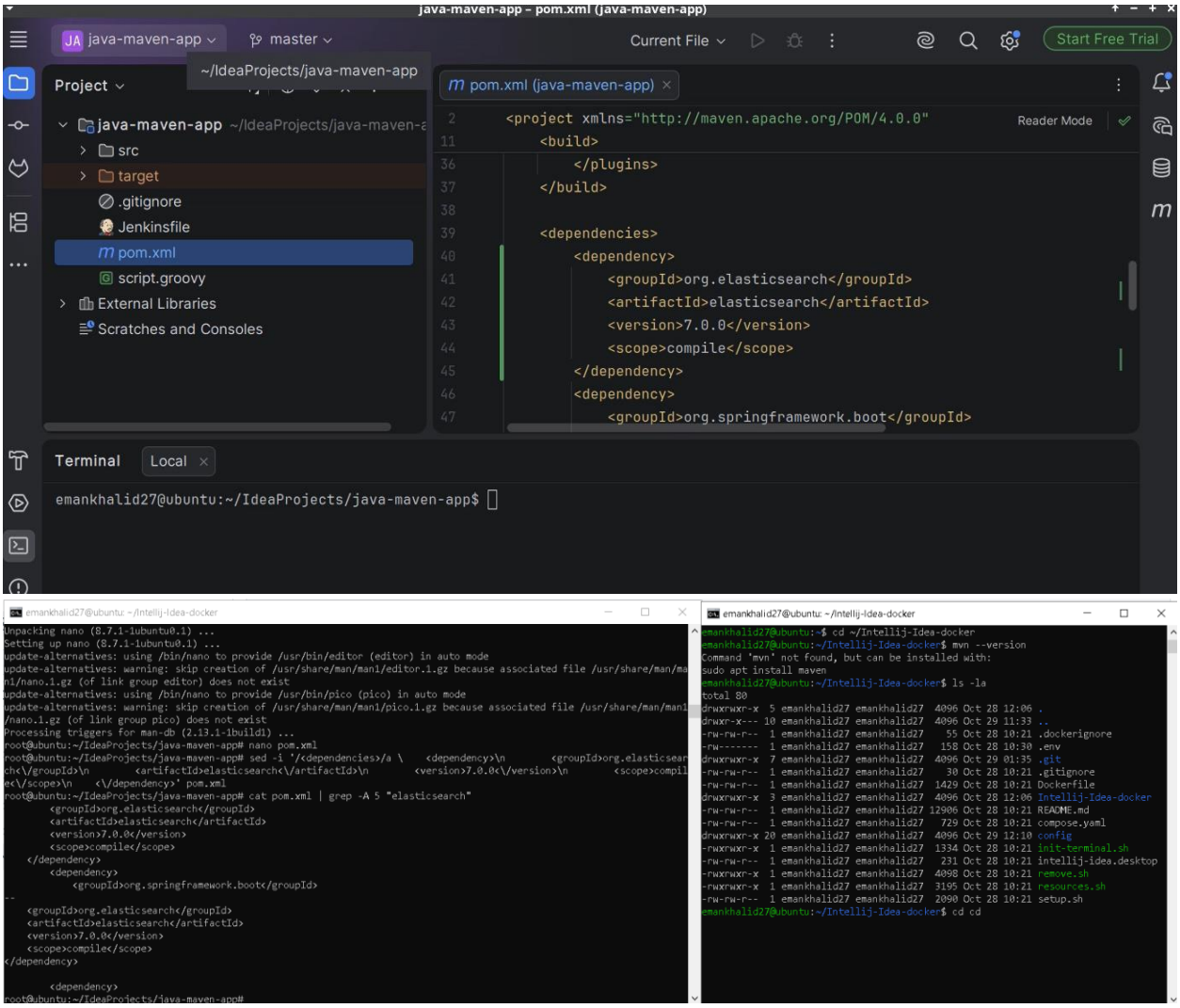


Figure 2.4: Elasticsearch dependency added to pom.xml. (maven_elasticsearch_dependency.png)

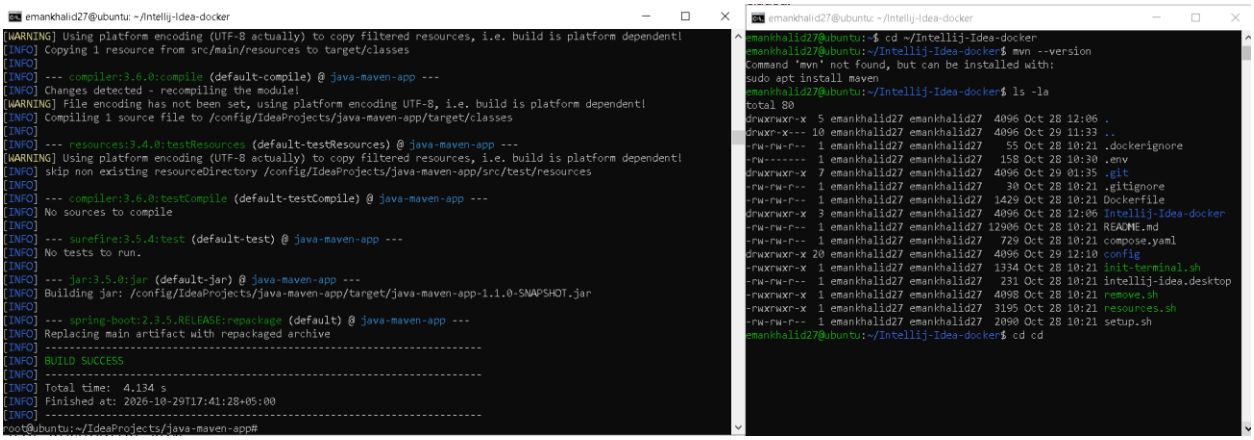


Figure 2.5: Successful repackage after dependency. (maven_repackage.png)

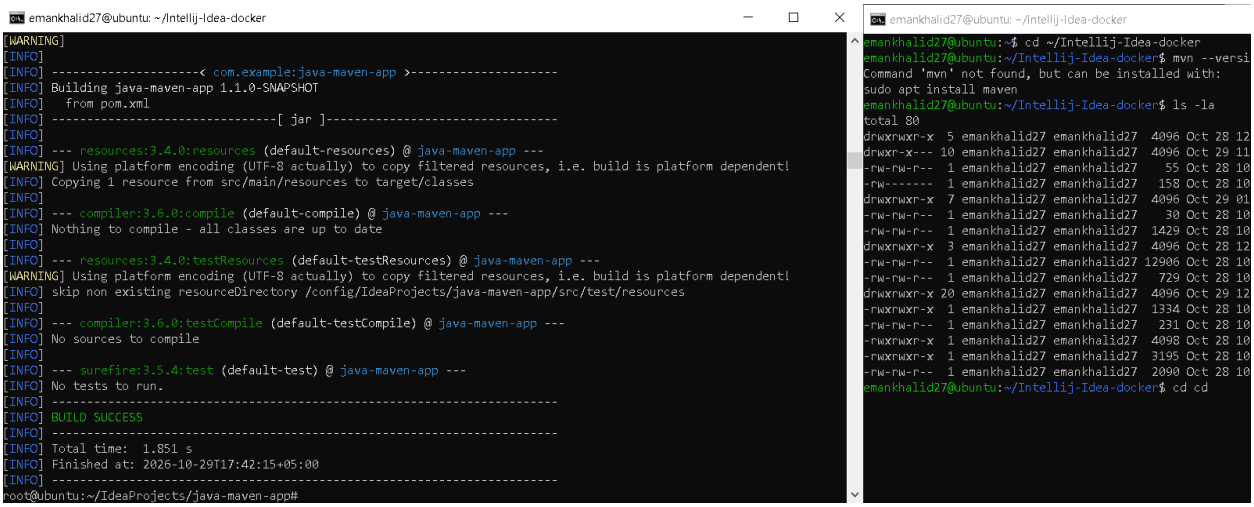


Figure 2.6: Maven test phase executed. (maven_test.png)

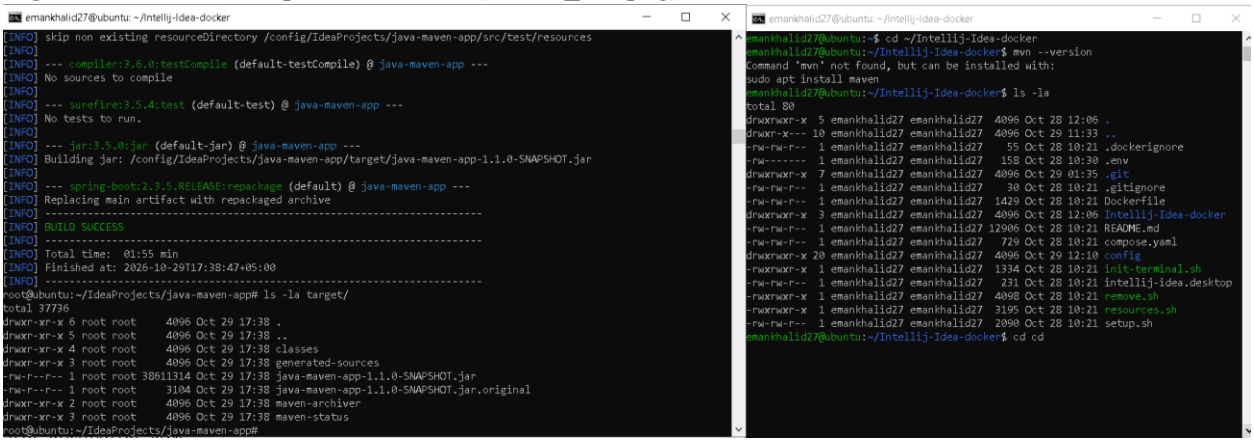


Figure 2.7: The generated JAR file in target/. (maven_jar.png)

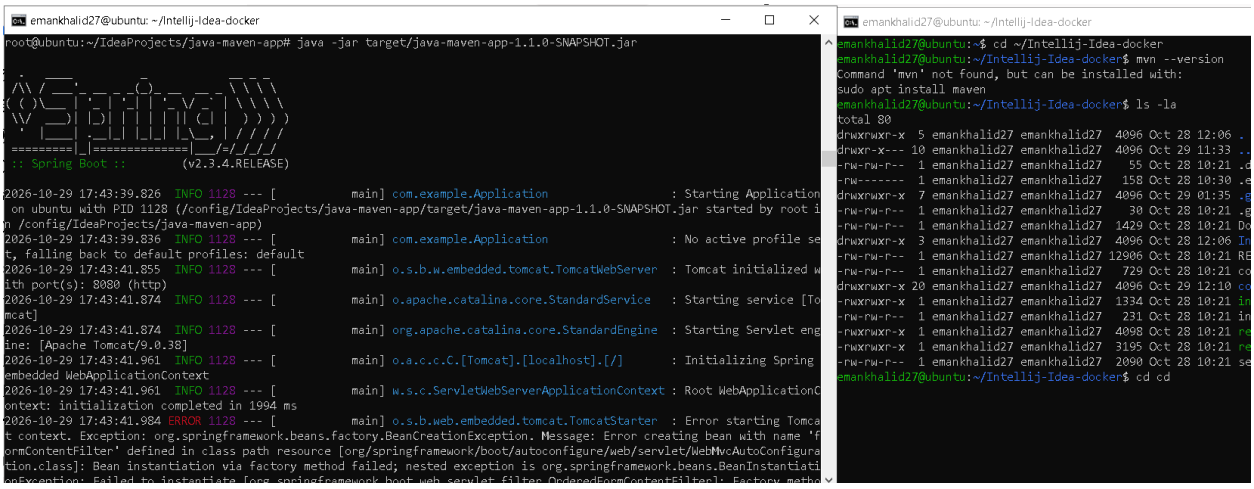


Figure 2.8: Running the JAR on port 8080. (maven_running.png)

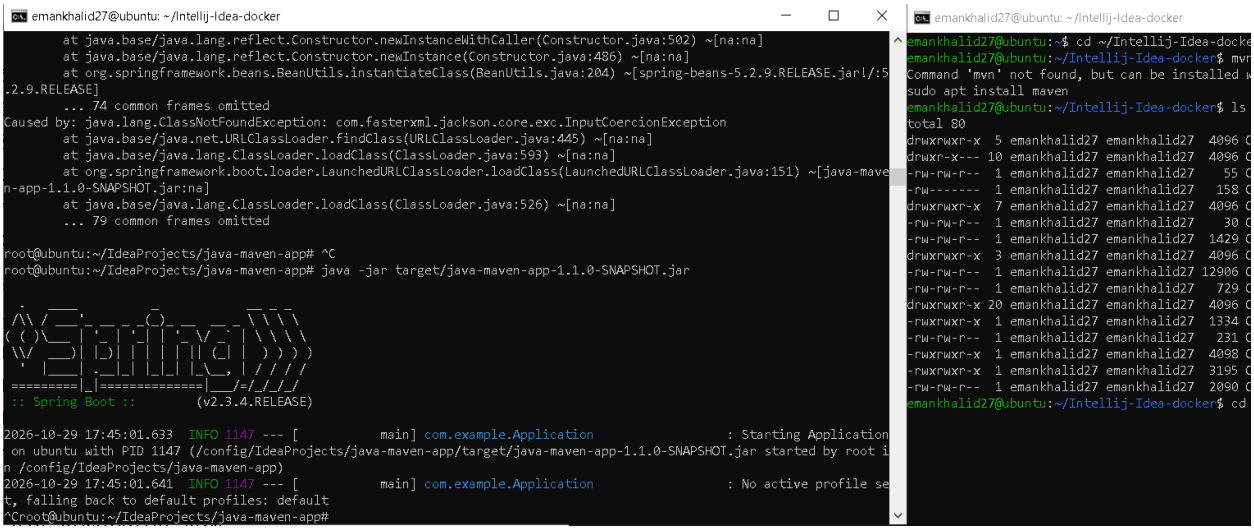


Figure 2.9: Stopping the JAR with ^C. (maven_stopped.png)

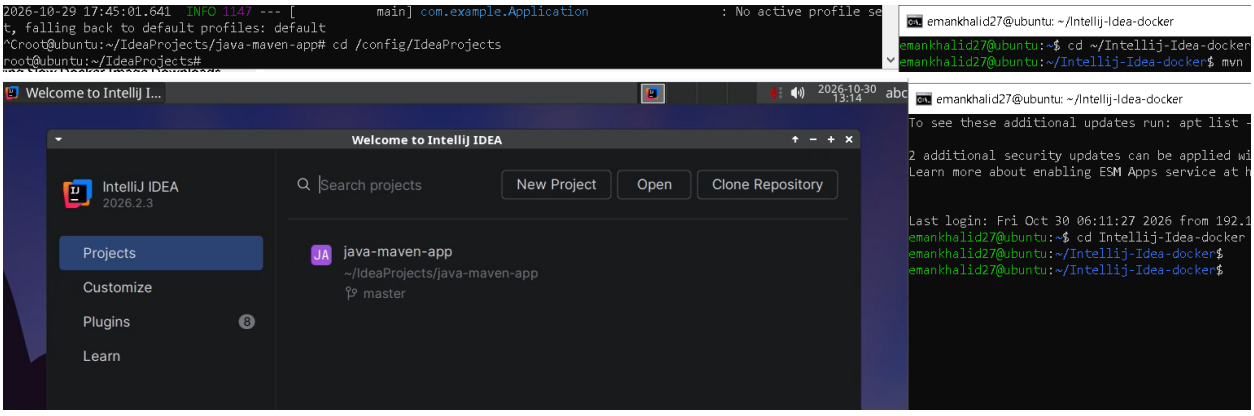


Figure 2.10: Closing the Maven project. (maven_project_closed.png)

Task 3: Build and Run the Gradle Project

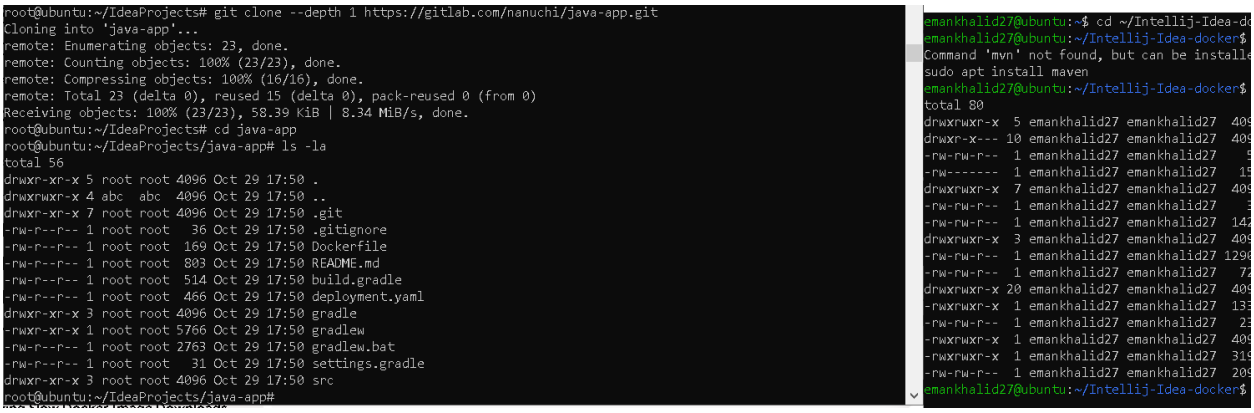


Figure 3.1: Cloning the Gradle project. (gradle_project_clone.png)

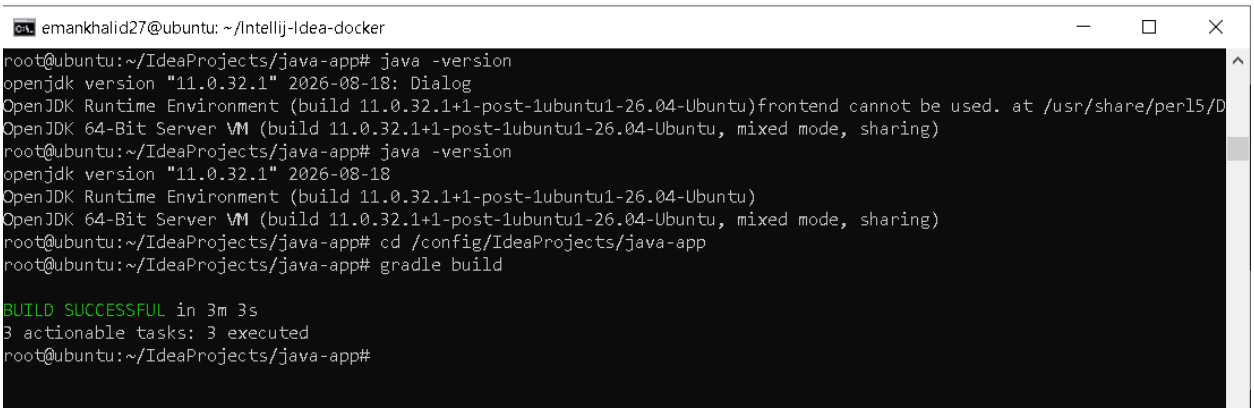


Figure 3.2: Successful Gradle build. (gradle_build.png)

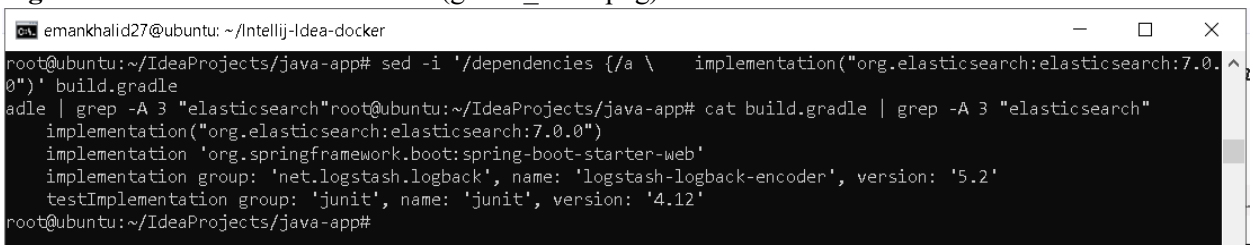


Figure 3.3: Elasticsearch dependency added to build.gradle. (gradle_elasticsearch_dependency.png)

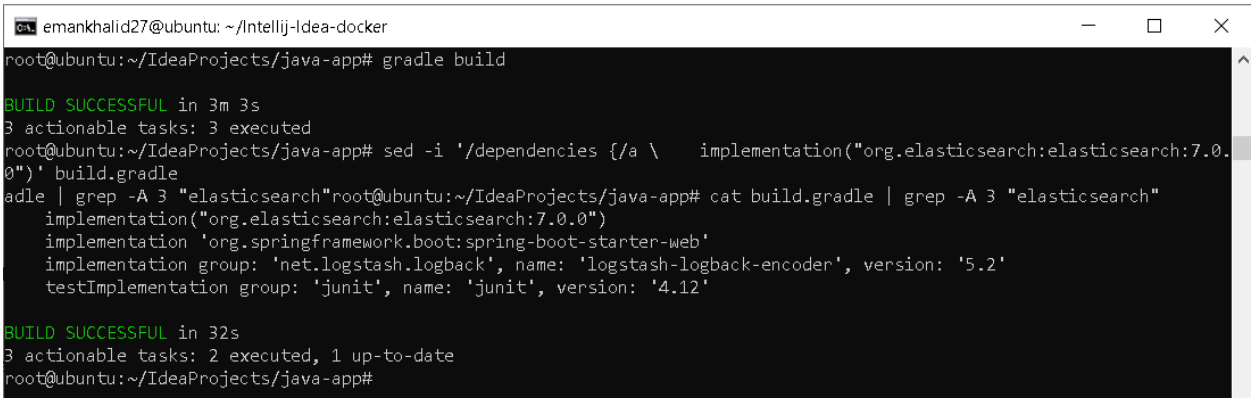


Figure 3.4: Successful Gradle rebuild. (gradle_rebuild.png)

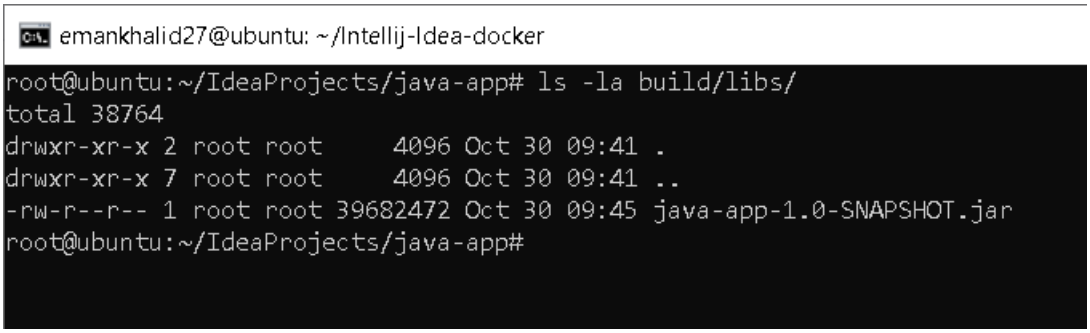


Figure 3.5: The generated JAR file in build/libs/. (gradle_jar.png)



Figure 3.6: Running the Gradle JAR. (gradle_running.png)

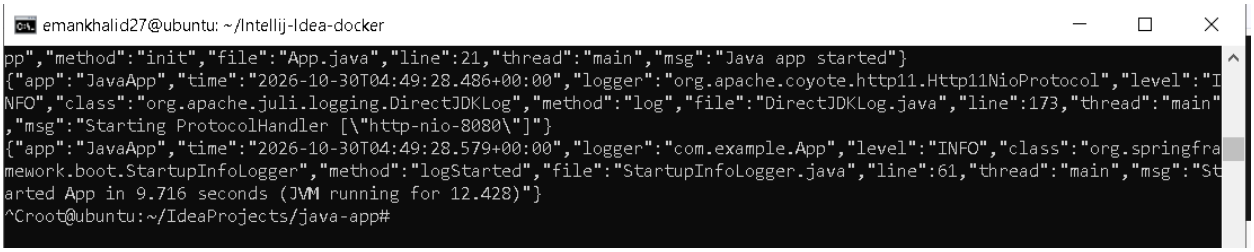
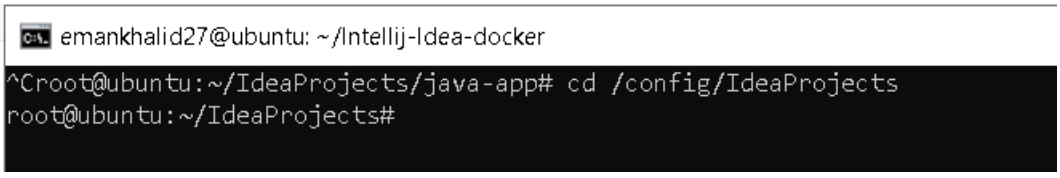


Figure 3.7: Stopping the JAR with ^C. (gradle_stopped.png)



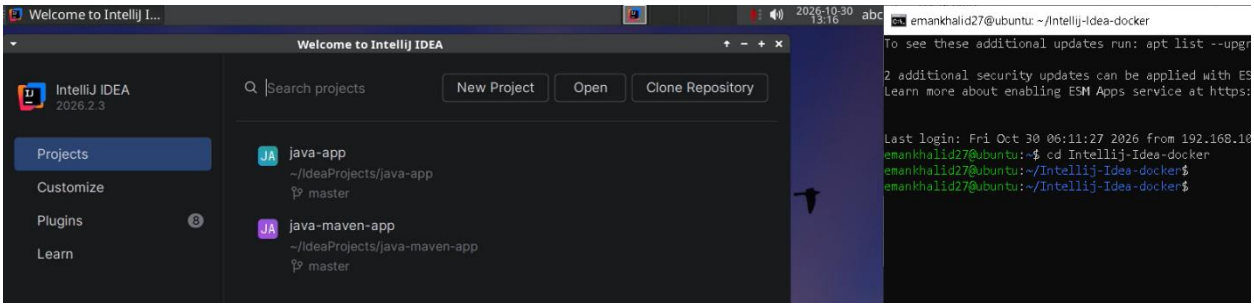


Figure 3.8: Closing the Gradle project. (gradle_project_closed.png)

Task 4: Install and Run the Node.js Project

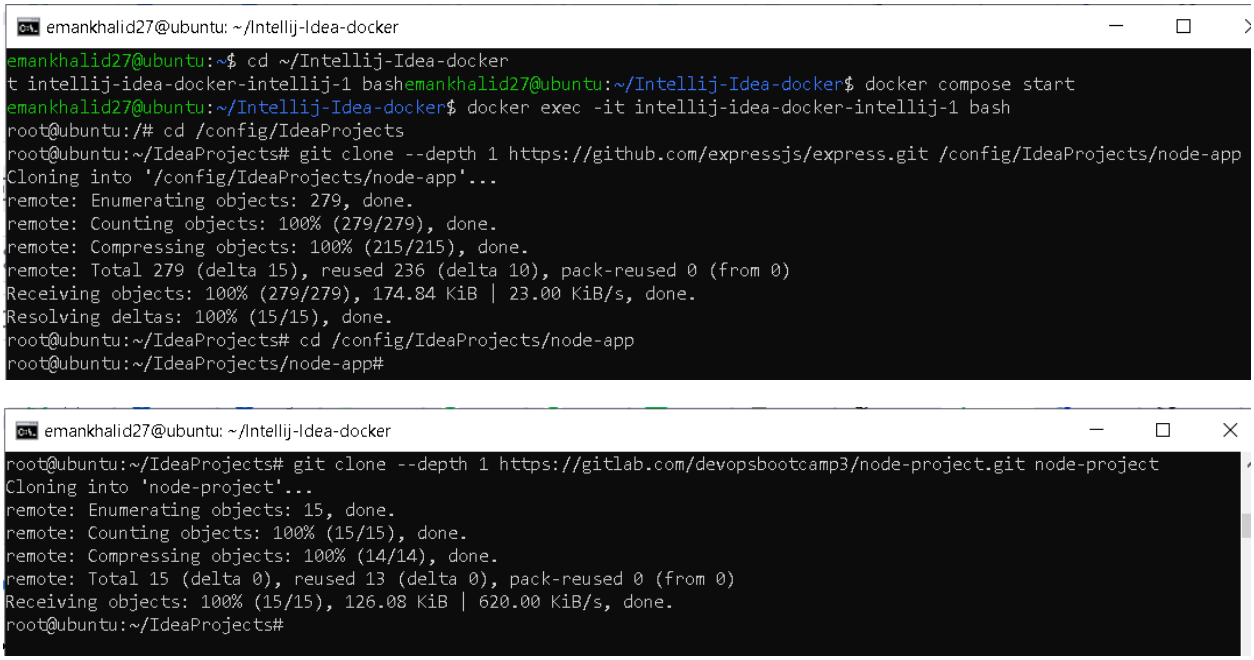


Figure 4.1: Cloning the Node.js project. (node_project_clone.png)

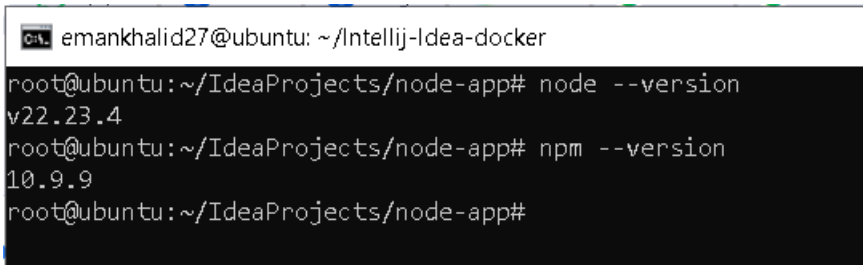


Figure 4.2: Node.js and npm versions installed. (nodejs_installed.png)

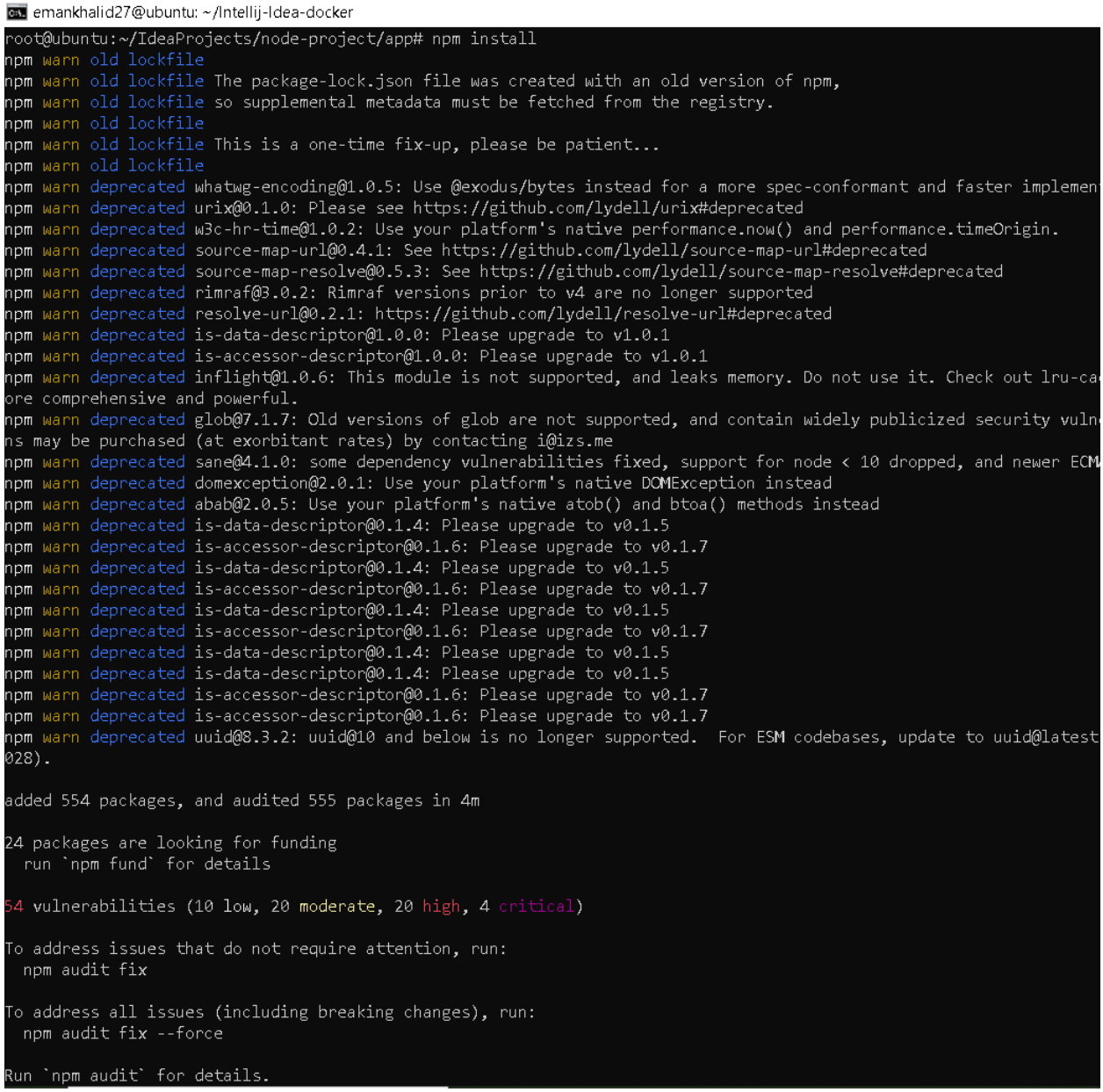


Figure 4.3: npm install completed. (npm_install.png)

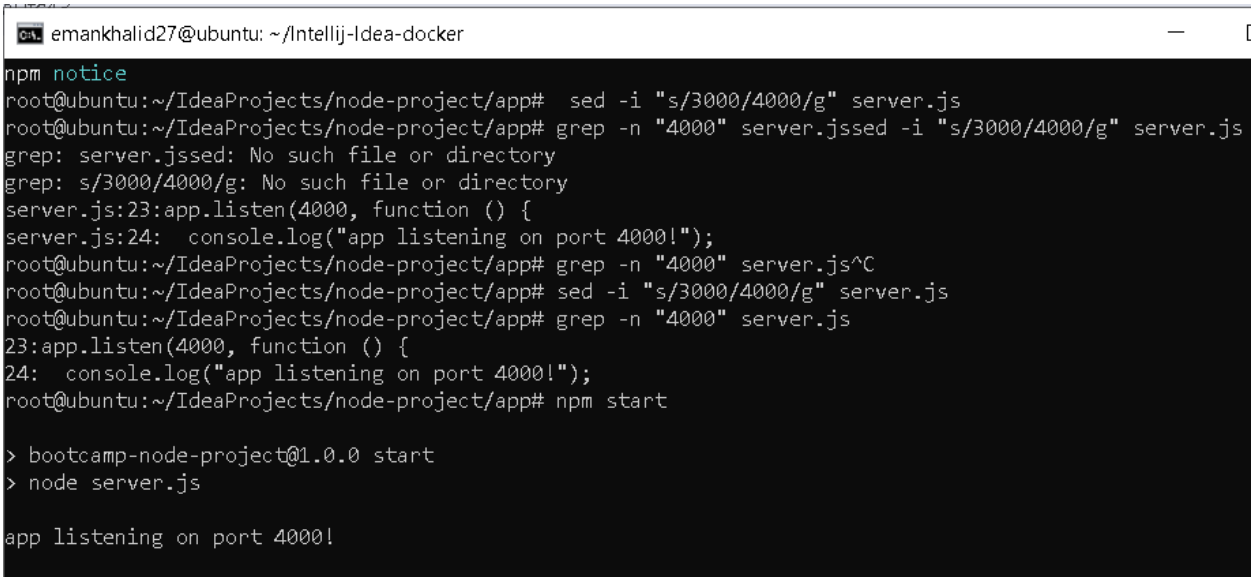


Figure 4.4: Node.js app running on port 4000. (node_running.png)

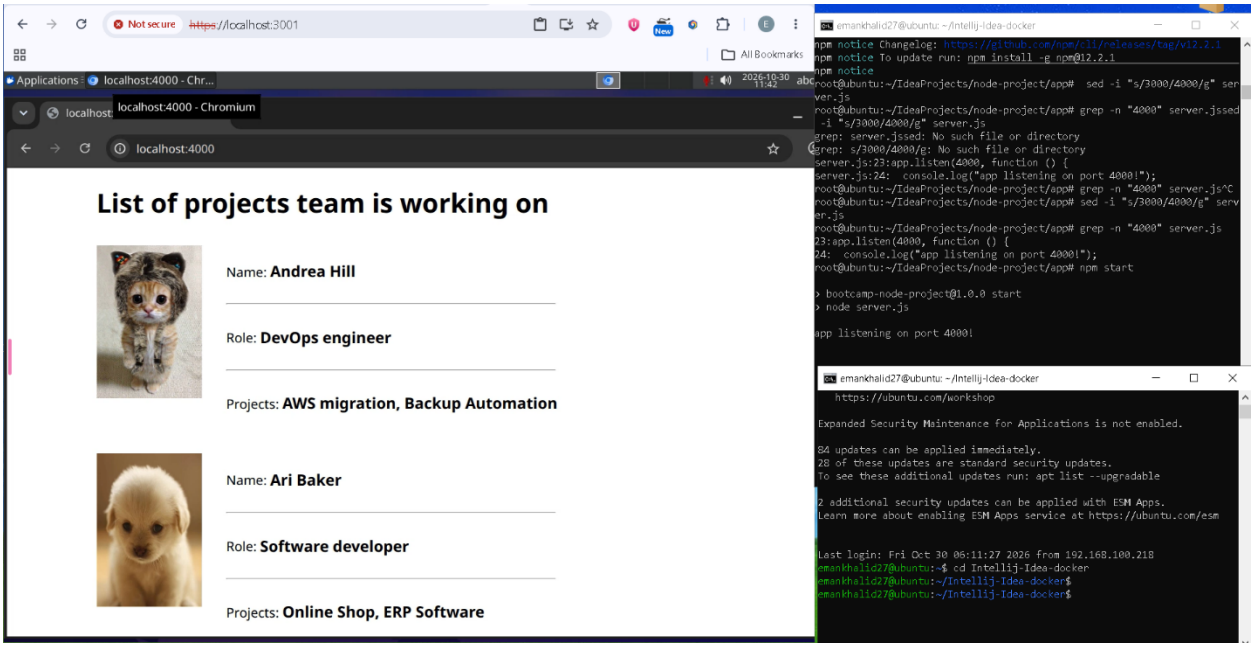


Figure 4.5: Viewing the app in the browser on port 4000. (node_browser.png)

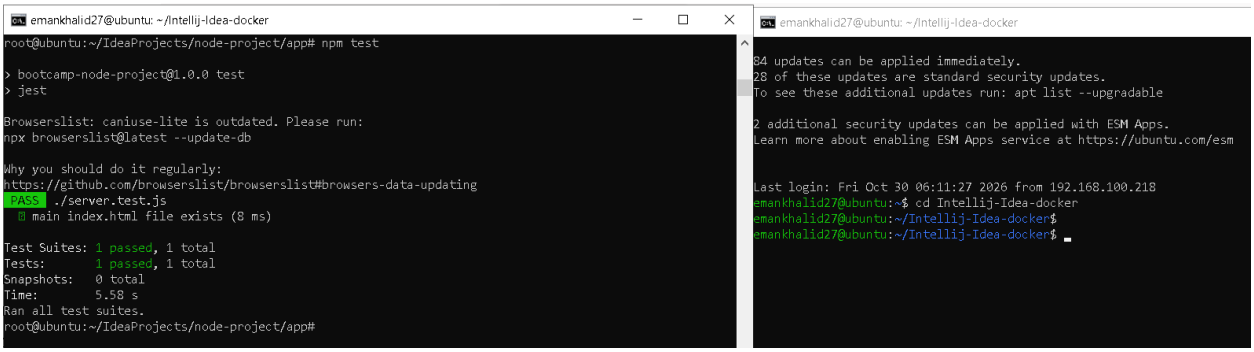


Figure 4.6: Jest test passed. (npm_test.png)

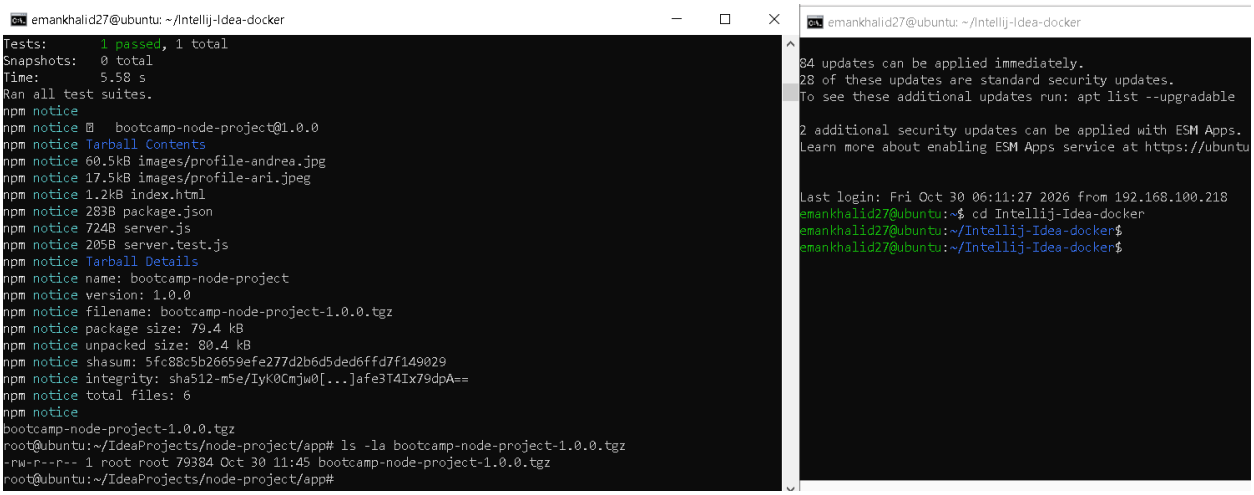


Figure 4.7: Creating the .tgz package. (npm_pack.png)

Task 5: Build the React and Node.js API Artifact

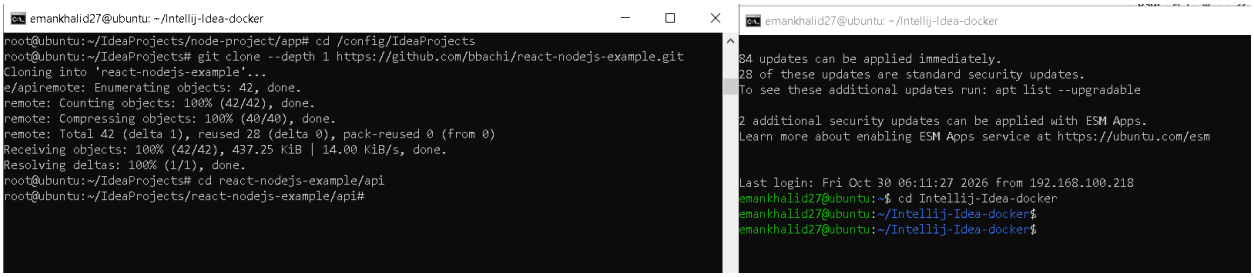


Figure 5.1: Cloning the React/Node project. (react_node_project_clone.png)

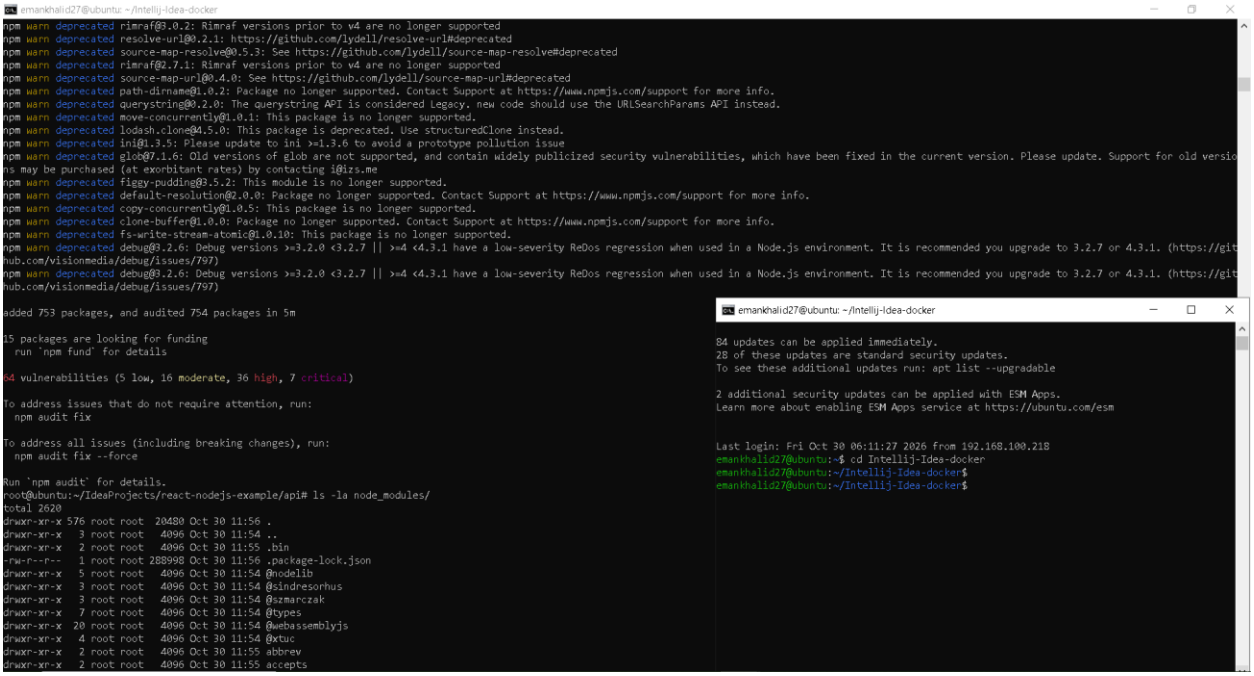


Figure 5.2: Installing API dependencies. (react_api_dependencies.png)

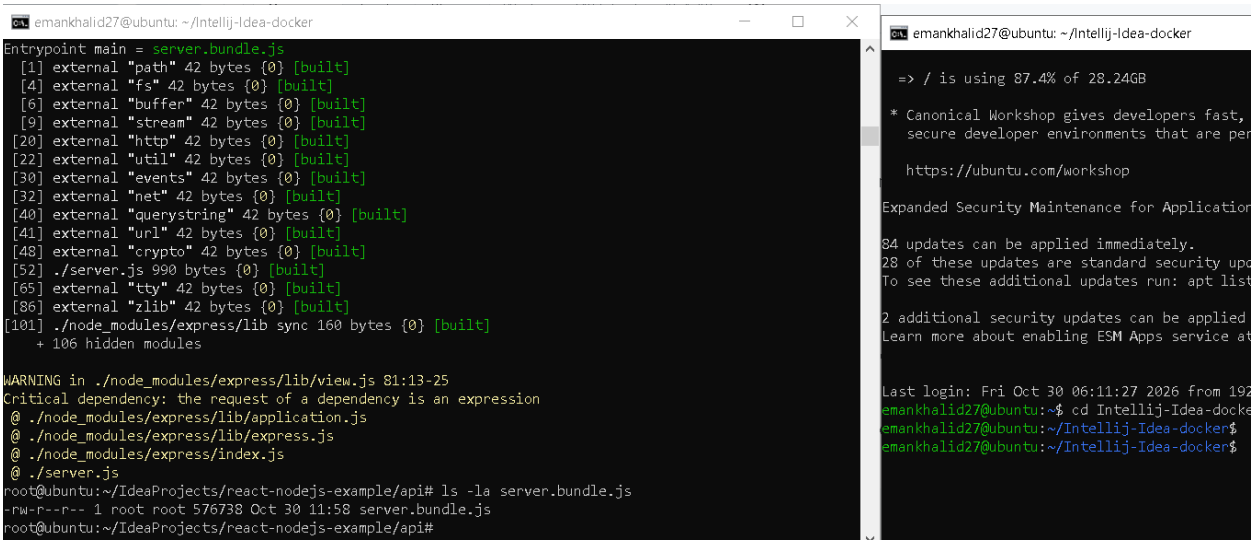


Figure 5.3: Successful webpack build creating server.bundle.js. (react_api_build.png)

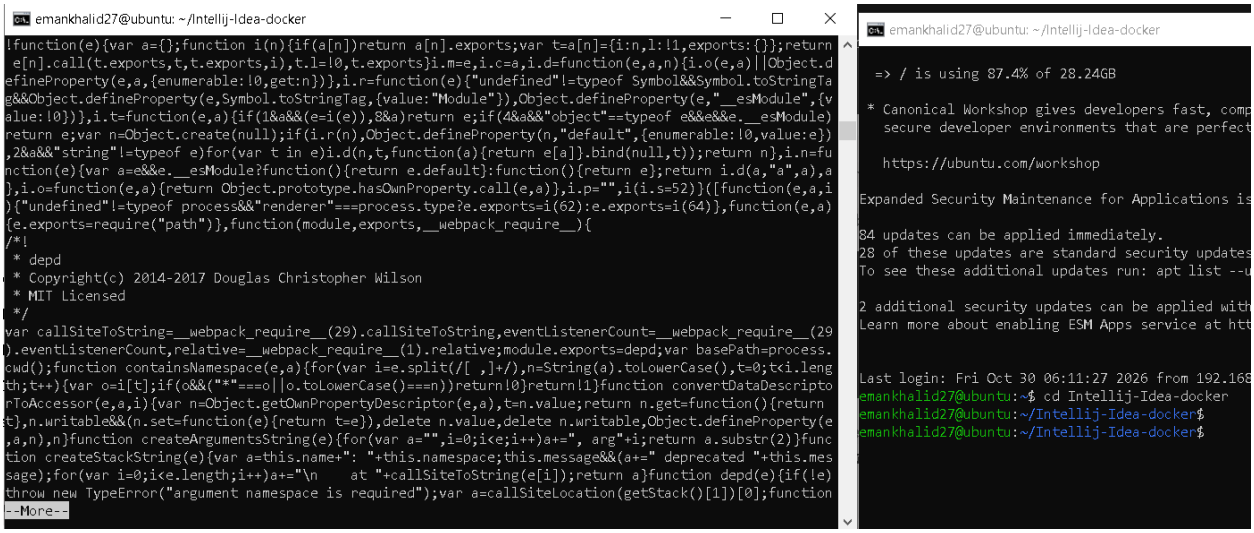


Figure 5.4: Viewing the bundle with more. (react_api_bundle.png)

Task 6: Build a Docker Image for the Node.js Project



Figure 6.1: Docker version installed. (docker_installed.png)



Figure 6.2: Verifying the Dockerfile exists. (dockerfile_exists.png)

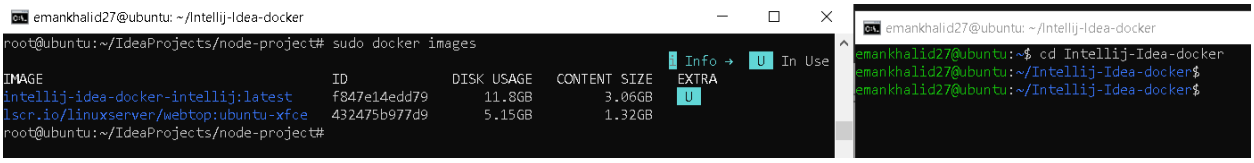


Figure 6.3: Docker images before the build. (docker_images_before.png)

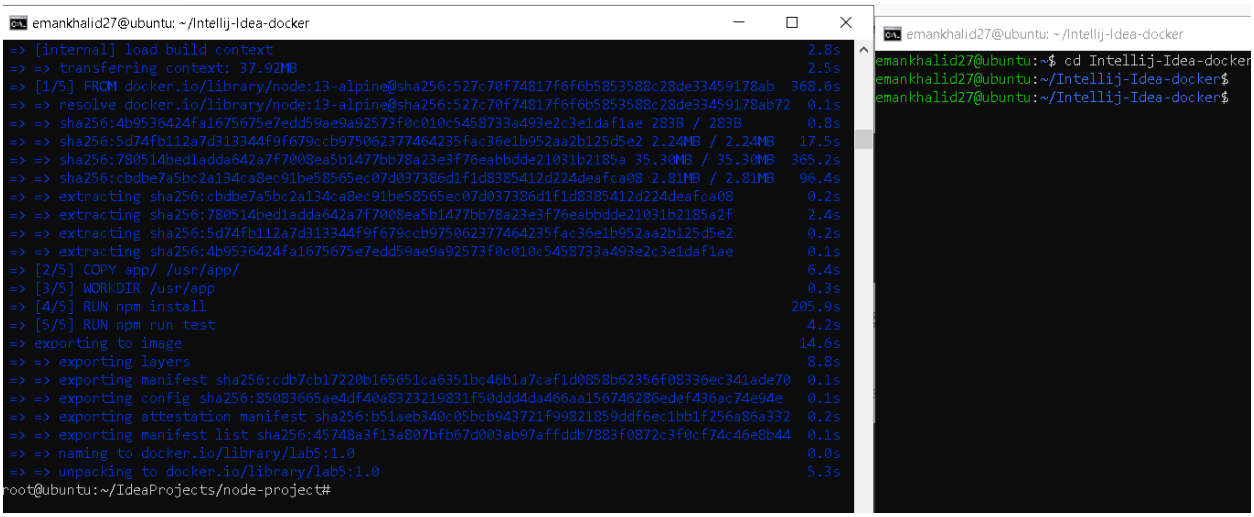


Figure 6.4: Building the image lab5:1.0. (docker_build.png)

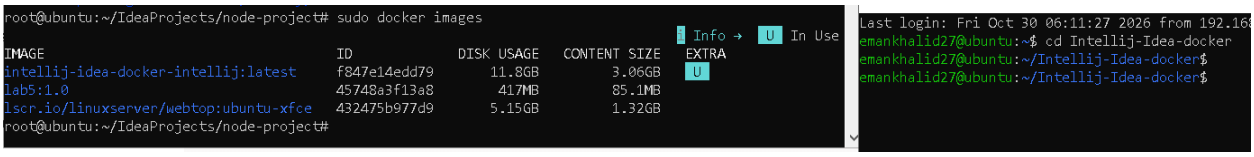
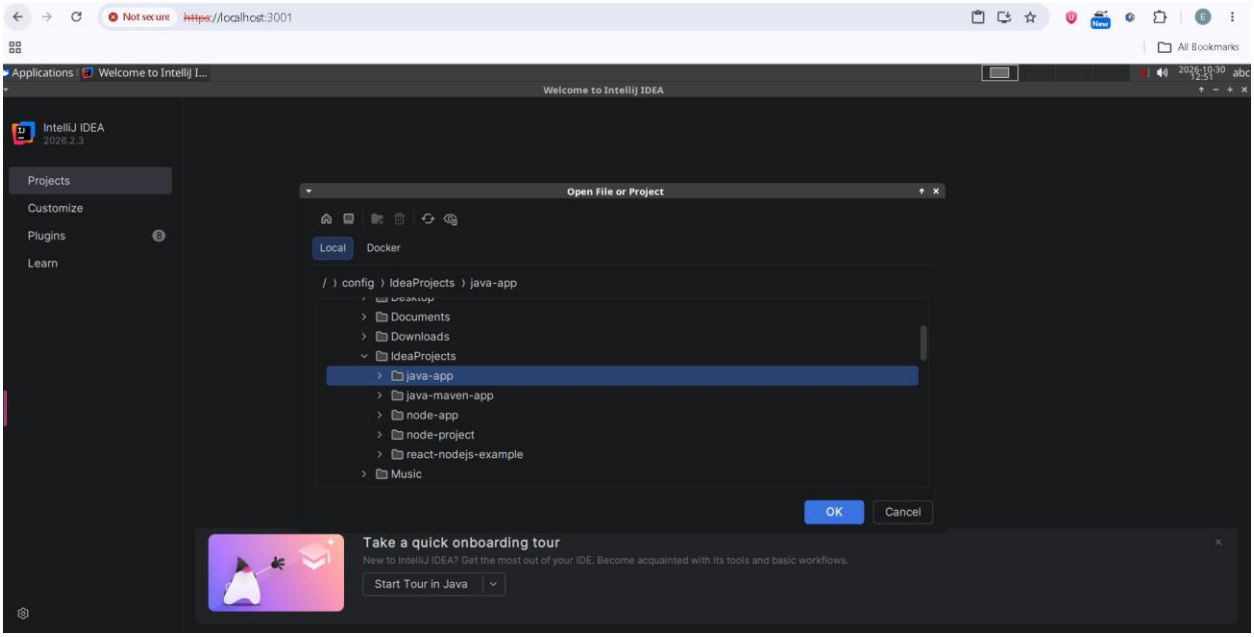


Figure 6.5: Docker images after the build, showing lab5:1.0. (docker_images_after.png)



4. Conclusion

I successfully cloned and launched the IntelliJ Docker environment. I used IntelliJ to build and run Maven, Gradle, Node.js, and React projects. I added dependencies, ran tests, created packages, resolved a port conflict, built a webpack bundle, and created a Docker image (lab5:1.0).